

Algoritmi e Strutture Dati – 23/08/2023 – Parte B**B1 – Rete idrica – Punti ≥ 8**

Siete responsabili di una rete idrica descritta da un grafo $G = (V, E)$. Il sottoinsieme $V_{in} \subseteq V$ contiene i nodi che sono sorgenti di acqua, e ogni nodo $v \in V_{in}$ produce $in(v)$ litri di acqua al secondo. Il sottoinsieme $V_{out} \subseteq V$ contiene i nodi che sono consumatori di acqua, con ogni nodo $v \in V_{out}$ che consuma $out(v)$ litri di acqua al secondo. Questi sottoinsiemi sono disgiunti: $V_{in} \cap V_{out} = \emptyset$. Gli altri nodi in V sono semplici distributori.

I nodi sono collegati da tubazioni descritte dall'insieme E ; la tubazione che collega un nodo u ad un nodo v ha una portata di $c(u, v)$ litri al secondo. Sfortunatamente, esiste un sottoinsieme $E_{per} \subseteq E$ di tubazioni che devono essere sostituite.

Per sostituire una tubazione, è necessario chiuderla; quando è chiusa, l'acqua non può più fluire attraverso di essa. Potreste domandarvi cosa succederebbe alla rete se chiudessimo una tubazione alla volta per ripararla. È possibile che alcuni nodi consumatori non ricevano abbastanza acqua?

Descrivere un algoritmo che prenda in input un grafo $G = (V, E)$, gli insiemi V_{in} , V_{out} e E_{per} , e restituisca **true** se chiudere una qualsiasi delle tubazioni guaste produce una riduzione di servizio per uno o più consumatori d'acqua, **false** altrimenti.

B2 – Sub-palindrome – Punti ≥ 10

Scrivere un algoritmo

`subPalindrome(item[] S, int n)`

che prenda in input una stringa S e stampi tutte le sottosequenze palindrome di S .

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale; se possibile, discutere un limite inferiore e superiore alla complessità.

Per esempio, la parola "casca" contiene le seguenti sottosequenze palindrome:

"", "c", "a", "s", "c", "a", "cc", "aa", "cac", "csc", "asa", "aca"

B3 – Stringhe eleganti – Punti ≥ 12

Una stringa binaria è detta *elegante* se è composta da un certo numero di 0 (possibilmente nessuno), seguito da un certo numero di 1 (anche qui, possibilmente nessuno).

Un'*inversione* di un bit della stringa consiste nel cambiare tale bit da 0 a 1 o da 1 a 0.

Scrivere un algoritmo

`int minInversions(item[] S, int n)`

che prenda in input una stringa S di dimensione n e restituisca il numero minimo di inversioni necessarie per renderla elegante.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio,

- 000010000 può essere trasformata nella stringa elegante 000000000 con 1 inversione (ma non con 0), quindi l'algoritmo deve restituire 1;
- 001110011 può essere trasformata nella stringa elegante 001111111 con 2 inversioni (ma non con 0 o 1), quindi l'algoritmo deve restituire 2.
- 111111111 e 000011111 sono già eleganti, quindi l'algoritmo deve restituire 0.